Middleware: anlamaq və müsahibədə izah etmək

Middleware ASP.NET Core-da HTTP sorğusu və cavabı üzərində işləyən komponentdir. Sorğunu qeyd etmək, istifadəçini tanımaq, icazəni yoxlamaq və xətaları tutmaq onun tipik işləridir. Middleware-lərin qoşulduğu ardıcıllıq **pipeline** adlanır.

Bu proqramı oxu. **Endpoint** sorğuya uyğun seçilən konkret işləyici koddur; burada /hello üçün yazılan hissədir.

C# - ayrıca Program.cs nümunəsi

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.Use(async (context, next) =>
{
    Console.WriteLine("A: evvel");
    await next(context);
    Console.WriteLine("A: sonra");
});

app.Use(async (context, next) =>
{
    Console.WriteLine("B: evvel");
    await next(context);
    Console.WriteLine("B: sonra");
});

app.MapGet("/hello", () =>
{
    Console.WriteLine("Endpoint");
    return "Salam";
});

app.Run();
```

Use çağırışları proqram başlayanda pipeline-ı qurur. İçlərindəki kod həmin hissəyə çatan HTTP sorğuları üçün işləyir. MapGet GET sorğusuna endpoint qeyd edir; qeydiyyat zamanı endpoint icra olunmur.

next: niyə çıxış tərsinə davam edir?

/hello sorğusu üçün yalnız öz yazdığımız qeydlərin sırası belədir:

Çıxış

```
A: evvel
B: evvel
Endpoint
B: sonra
A: sonra
```

HttpContext cari sorğu və cavabın məlumatlarını saxlayan obyektdir. Request gələn sorğunu, Response hazırlanan cavabı, User istifadəçi məlumatını verir. Məsələn, Request.Path sorğunun yoludur.

next pipeline-ın qalan hissəsini çağıran funksiyadır. **await next(context)** onu işlədir, tamamlanmasını gözləyir, sonra qaldığı sətirdən davam edir. A, B-ni; B isə endpoint-ə gedən davamı gözləyir. Buna görə bitmə sırası B, sonra A-dır. Middleware-lər yenidən əvvəldən çağırılmır.

Short-circuit davamı çağırmadan sorğunu həmin mərhələdə cavablandırmaqdır. Birinci nümunədə B-nin next sətrindən əvvəl bunu əlavə et:

C# - B middleware-inin içində

```
if (context.Request.Path == "/blocked")
{
    context.Response.StatusCode = 403;
    await context.Response.WriteAsync("Icaze yoxdur");
    return;
}
```

403 bu nümunədə girişə icazə verilmədiyini bildirir. /blocked üçün A əvvəl, B əvvəl, A sonra görünür; endpoint və B sonra işləmir. **B davamı kəsir, amma onu gözləyən A normal tamamlanmadan sonra davam edə bilir.**

next-i await etmədən çağırmaq davam bitməmiş sonrakı kodun işləməsinə səbəb ola bilər. Sorğunu bir neçə dəfə işlətməmək üçün next-i normal axında bir dəfə çağır.

Use, Run, Map və işləmə sırası

Yazılış	Nə edir?
app.Use(...)	Middleware əlavə edir; next ilə davam edə və ya cavab verib davamı kəsə bilər.
app.Run(context => ...)	next almayan sonlandırıcı işləyicidir. Ondan sonrakı əsas pipeline hissəsinə keçmir.
app.Run()	Tətbiqi işə salır. Yuxarıdakı Run ilə fərqli çağırışdır.
app.Map("/admin", branch => ...)	Yol uyğun gələndə ayrıca pipeline qoluna keçir; həmin qol əsas pipeline-ın sonrakı hissəsinə qoşulmur.
app.UseWhen(...)	Şərtə uyğun qol yaradır; qol davamı kəsməzsə əsas pipeline-a birləşir.
app.MapGet("/hello", ...)	Yol və GET metodu üçün endpoint qeyd edir; Map ilə qol yaratmaq nümunəsi ilə eyni iş deyil.

Tipik məntiqi ardıcıllıq:

Sorğunun yolu

```
Xeta idaresi -> Routing -> Authentication
             -> Authorization -> Endpoint
```

Routing uyğun endpoint-i seçir. **Authentication** istifadəçinin kimliyini müəyyən edir. **Authorization** həmin istifadəçinin əməliyyata icazəsini yoxlayır. İstifadəçini tanımaq onu avtomatik hər yerə buraxmaq demək deyil; endpoint üçün uyğun icazə qaydası da tələb olunur.

Authentication authorization-dan əvvəl yerləşir. Xəta idarəsi əvvəldədir ki, davamındakı xətaları tuta bilsin. Cavab verib dayandıran komponentdən sonra yerləşən hissə həmin sorğu üçün işləməyə bilər.

Minimal API-də WebApplication bəzi routing və təhlükəsizlik mərhələlərini uyğun qeydiyyatlar olduqda avtomatik qoşa bilər. Yuxarıdakı yol məntiqi ardıcıllıqdır; hər layihədə bütün həmin metodları əl ilə yazmaq məcburiyyəti kimi oxuma.

Xəta, finally və cavabın başlanması

Endpoint xəta atsa və aşağıda tutulmasa, xəta onu çağıran middleware-ə ötürülür. await next-dən sonrakı adi sətir bu halda işləməyə bilər. Uğurlu və xətalı çıxışda mütləq yerinə yetiriləcək işi finally-də saxla:

C# - middleware-in içində

```
try
{
    await next(context);
}
finally
{
    Console.WriteLine("Sorgunun icrasi bitdi");
}
```

Exception proqramın adi icrasını pozan xətadır. **catch** xətanı tutmaq, **finally** isə həmin blokdan çıxanda işi tamamlamaq üçündür. finally xətanı öz-özünə uğurlu cavaba çevirmir.

UseExceptionHandler kimi middleware davamında idarə olunmamış xətanı uyğun xəta cavabına çevirmək üçün işlədilir. Özündən əvvəlki middleware-in xətasını tuta bilməz. Hər HTTP 400 və ya 404 cavabı exception demək deyil.

Header - başlıq cavabın Content-Type kimi əlavə məlumatıdır. **Body - gövdə** isə göndərilən məzmundur. Cavab göndərilməyə başlayıbsa status və başlıqları dəyişmək xəta verə bilər; Response.HasStarted bunu göstərir.

Ona görə next-dən sonra log yazmaqla cavabın başlığını dəyişmək eyni dərəcədə təhlükəsiz işlər deyil. Gövdəyə təsadüfi mətn əlavə etmək də JSON kimi cavab formatını poza bilər. JSON məlumat ötürmək üçün mətn formatıdır.

Başlığı göndərilməzdən əvvəl dəyişmək üçün onu uyğun mərhələdə əvvəlcədən təyin et və ya Response.OnStarting ilə başlıqların göndərilməsindən əvvəl işləyəcək funksiyanı qeyd et. Pipeline-da geri davam etmək cavabın hələ başlamadığına zəmanət vermir.

Öz middleware-in və DI ilə düzgün işləmək

RequestDelegate HttpContext qəbul edib Task qaytaran davam funksiyasının tipidir. Task tamamlanması gözlənilən işi təmsil edir. Adi middleware sinfində davamı konstruktorda alıb InvokeAsync daxilində çağırırıq:

C# - ayrı RequestLogMiddleware.cs faylı

```
public class RequestLogMiddleware
{
    private readonly RequestDelegate _next;

    public RequestLogMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        Console.WriteLine(context.Request.Path);
        await _next(context);
    }
}
```

Program.cs-də endpoint-lərdən əvvəl `app.UseMiddleware<RequestLogMiddleware>();` ilə əlavə edilir. Public Invoke və ya InvokeAsync metodu Task qaytarmalı, ilk parametr kimi HttpContext almalıdır.

Bu adi yanaşmada middleware pipeline qurulanda yaradılır, hər sorğuda yenidən yaranmır. Eyni obyekt paralel sorğulara xidmət edə bilər. Sorğuya aid məlumatı sinfin dəyişənlərində və static dəyişəndə saxlama; lokal dəyişəndə və ya cari context-də saxla.

Scoped servis bir scope daxilində paylaşılan obyektdir. Adi HTTP sorğusunda həmin scope cari sorğuya aiddir. DbContext kimi scoped servisi bu middleware-in konstruktoruna götürmək əvəzinə InvokeAsync parametrindən al:

C# - AppDbContext layihənin öz baza context-idir

```
public async Task InvokeAsync(
    HttpContext context, AppDbContext db)
{
    await _next(context);
}
```

İstisna: IMiddleware yanaşmasında middleware scoped/transient kimi DI-də qeydiyyatdan keçib sorğu üzrə aktivləşdirilə bilər; onda scoped servis konstruktorda da alına bilər. Buna görə bütün middleware-ləri singleton adlandırmaq dəqiq deyil.

Müsahibədə qarışdırılan suallar

Middleware ilə MVC filter fərqi? Middleware HTTP pipeline səviyyəsindədir. MVC filter controller action-ının icra mərhələlərində işləyir. Action controller-də konkret sorğunu işləyən metoddur. Bütün sorğuların müddətini ölçmək middleware, action arqumentlərini yoxlamaq action filter üçün nümunədir.

Hər middleware hər sorğuda işləyir? Yalnız sorğu həmin hissəyə çatırsa. Qol seçilməsi və ya əvvəlki short-circuit ona çatmağa mane ola bilər.

next çağırılmasa controller işləyər? Controller hələ aşağıdadırsa ona çatılmır. Middleware özü cavab verməlidir; heç nə yazmadan bitirmək də uğurlu biznes əməliyyatı yerinə keçmir.

Constructor ilə InvokeAsync nə vaxt işləyir? Adi middleware-in konstruktoru pipeline qurularkən, InvokeAsync isə həmin middleware-ə çatan sorğularda işləyir. IMiddleware üçün aktivləşmə yanaşması fərqlidir.

Niyə əvvəl A sonra B, cavabda B sonra A? Hər biri çağırdığı davamın bitməsini gözləyir, sonra qaldığı yerdən davam edir. Ayrı ikinci pipeline qurulmur.

Uyğun endpoint tapılmasa nə olur? Başqa komponent cavablandırmırsa son mərhələdə 404 qaytarılır. Bu, mütləq exception olması demək deyil.

Middleware hansı pattern-ə oxşayır? Chain of Responsibility: komponent işi görüb davamı ötürür və ya zənciri kəsir. ASP.NET Core əlavə olaraq davamı çağırışın əvvəlində və sonrasında kod işlətməyə imkan verir.

Rəsmi istinadlar: middleware və pipeline, öz middleware-ini yazmaq, IMiddleware, HttpContext, routing, authentication, MVC filters.